XGBoost Validation Strategy: ProGati Festive Forecast

Oct 29, 2026 · @Yash

Objective

This validation decides whether the enquiry and call lag features improve the XGBoost Sep–Dec forecast, and measures how accurate the model would have been in past festive seasons.

It compares three variants on six past folds, scored by WMAPE and bias. Hyperparameter tuning comes after this comparison, not before. The forecast to be produced is Sep–Dec 2026 (122 days), with the model trained to 31 Aug 2026.

Fold design

The six folds use an expanding window: training always starts on 1 Apr 2023 and ends the day before the test window. Test windows are consecutive four-month blocks, so every day from 1 Sep 2024 to 31 Aug 2026 is tested exactly once. The seventh row is the production run, used only as a final check.

Fold	Train period	Train days	Test period	Test days	Festivals in test window
F1	1 Apr 2023 → 31 Aug 2024	519	1 Sep → 31 Dec 2024	122	Navratri 3 Oct, Diwali 1 Nov
F2	1 Apr 2023 → 31 Dec 2024	641	1 Jan → 30 Apr 2025	120	none
F3	1 Apr 2023 → 30 Apr 2025	761	1 May → 31 Aug 2025	123	none
F4	1 Apr 2023 → 31 Aug 2025	884	1 Sep → 31 Dec 2025	122	Navratri 22 Sep, Diwali 20 Oct
F5	1 Apr 2023 → 31 Dec 2025	1,006	1 Jan → 30 Apr 2026	120	none
F6	1 Apr 2023 → 30 Apr 2026	1,126	1 May → 31 Aug 2026	123	none

Fold	Train period	Train days	Test period	Test days	Festivals in test window
Final check	1 Apr 2023 → 31 Aug 2026	1,249	1 Sep → 31 Dec 2026	122 (about 55 with actuals as of 25 Oct 2026)	Navratri 11 Oct, Diwali 8 Nov

F1 and F4 are the festive folds. F1 has seen only one festive season in training (2023), so expect a weaker score there. The final-check fold is never used to choose a variant or a setting.

Variants

Three variants run on every fold. Only variant C is a real candidate; variant B exists to measure how much the unmasked lags leak.

Variant	Definition	Use
A	`reg:tweedie`, no lag features	Baseline
B	A plus the enquiry and call lag columns exactly as they sit in the table	Diagnostic only: shows the size of the leak. Never used for a decision
C	A plus the same lag columns, masked by the rule below	The real test of the lag features

Masking rule for C. A lag of k days on date d is kept only if d − k is before the fold's origin; otherwise the value is set to blank.

- Test rows: the origin is the test start date (for example 1 Sep 2025 for F4).
- Training rows: the origin is the start of the four-month block containing the row (1 Jan, 1 May or 1 Sep), so training and test see the same blank pattern.
- Effect: in a 122-day window, a 7-day lag is usable on about 7 days, a 14-day lag on about 14 and a 21-day lag on about 21. The columns will be mostly blank by design.

The lag columns are `ENQUIRY_LAG_7`, `ENQUIRY_LAG_14`, `ENQUIRY_LAG_21` and the matching `UNIQUE_CALLS_LAG_*` columns.

Settings held fixed

Every one of the 18 fits (6 folds × 3 variants) uses identical settings, so the only difference between variants is the feature set.

- Objective `reg:tweedie`, `tweedie_variance_power` 1.5.
- `subsample` = 1.0 and `colsample_bytree` = 1.0, so runs are close to deterministic and no extra seed runs are needed.
- One fixed seed (`random_state`), data sorted deterministically by series and date.
- A fixed number of trees; no early stopping on any test fold.
- No sample weights at this stage, so the variants stay clean. Festive weights are tested in the next stage.
- The same series filter and the same non-lag features in every fold.

Metrics and records

Every fit is scored on its test window with two numbers, because WMAPE alone hides whether the model is systematically low.

- **WMAPE** = sum of |forecast − actual| ÷ sum of actual. Errors do not cancel across days or series.
- **Bias** = (sum of forecast ÷ sum of actual) − 1. A negative value means under-prediction, which matters for SOQ because it means ordering too little.

Recorded for every fit: WMAPE and bias per fold, festive folds (F1, F4) reported separately from the off-season folds, and WMAPE by horizon band (days 1–21, 22–60, 61–122). The horizon bands show where, if anywhere, the lag features help.

The comparison is read as a paired difference per fold (variant minus A), not as an average across folds.

Leakage controls

A backtest must behave as if it stood on the fold's origin date. Anything the model or its features could not have looked up on that date is leakage.

1. **Zero-the-test-actuals test.** On one fold, replace the actual sales in the test window (and the calls and enquiries columns in that window) with zeros or random values. Rebuild the features from the changed data, retrain with the same seed and forecast. The two forecasts must be identical; if not, some feature reads the test period. Compare forecasts, not scores, and allow a tiny numeric tolerance.
2. **Masking check.** Confirm that in variant C a 7-day lag is non-null on only about 7 days of each test window, and likewise for the 14- and 21-day lags.

3. **Data start dates.** Confirm when the calls and enquiries columns begin. If they start after April 2023, the early folds are not comparable with the later ones.
4. **Seed and determinism.** Fix `random_state`, sort rows deterministically, and for the leak test use `n_jobs=1` on the CPU, since multithreading and GPUs can add tiny numeric differences.
5. **No tuning on test folds.** No setting is chosen by looking at a test fold's score during this comparison.

Decision rules

Keep the lag features only if variant C clearly beats variant A; otherwise drop them and use the simpler model. The thresholds below are proposals and can be changed.

- **Leak size:** B against A. A large gain that is uniform across all six folds, including off-season ones, is the signature of leakage and is not a result.
- **Keep the lags** if C beats A in at least 5 of 6 folds, including both festive folds (F1, F4), with a mean WMAPE gain of about 1 point or more.
- **Treat as a tie** if the mean gain is below the run-to-run noise. If subsample and colsample are 1.0, measure noise by retraining A with 3 seeds on one or two folds.
- **Read bias with WMAPE.** A variant with lower WMAPE but a larger festive under-prediction is not an improvement for SOQ.
- Expect any real lag gain to sit in the first 3 weeks of the window, because lags of 7–21 days cannot inform later days.

Next steps

Once the lag decision is made, work on festive under-prediction before hyperparameters, because bias in the festive folds is expected to matter more than any tuning gain.

1. **Diagnose the festive bias** by month, model family, dealer size and days around Navratri and Diwali.
2. **Test targeted fixes** one at a time against the baseline on the same folds: festive sample weights, `tweedie_variance_power` (1.2, 1.5, 1.8), spend and adstock features (only if the spend plan was known on each fold's origin date), and a feature for the year's overall level.

3. **Tune hyperparameters with Optuna** on Pitru Paksha-aligned festive folds (origins about 12 Aug 2025 and about 23 Aug 2024, each with a 122-day test window). Tune on one, confirm on the other, include festive bias in the objective, and measure seed noise if subsample or colsample go below 1.0.
4. **Train the final model** on 1 Apr 2023 → 31 Aug 2026 with the chosen setup and score it on the final-check fold, using the days that have actuals.

Limits and open items

The design has two festive folds, so small differences in WMAPE between variants are not conclusive.

- F1 trains on one festive season only and will score worse than F4 for that reason alone.
- No fold covers 2023, because the data starts on 1 Apr 2023.
- The final-check fold has actuals for only about 55 of its 122 days as of 25 Oct 2026.
- The festival dates used for the fold table are the Navratri and Diwali dates already used in the TFT pipeline; the Pitru Paksha start dates for 2024 and 2026 should be read from the PP flag column, not typed by hand.
- Open: start dates of the calls and enquiries columns, and whether the spend plan for each fold was known on its origin date.
- Illustrative numbers used while designing this plan were made up and are not results.